Tracks: Integer Programming Formulation and Resolution Approach

CHACON Jose
RO03, Games, Graphs and Operations Research
ENSTA Paris
Palaiseau, France

FLORIDO Santiago
RO03, Games, Graphs and Operations Research
ENSTA Paris
Palaiseau, France

***Abstract*—This report presents a graph-based mixed-integer formulation and Python implementation for the *Tracks* logic puzzle. A rectangular puzzle grid is modeled as an undirected grid graph, where cells are vertices and admissible track continuities are edges. The puzzle is treated as a feasibility problem: the objective is to find a single railway path between two prescribed terminals while satisfying row and column clues, fixed hints, local degree rules, and global connectedness. The formulation uses binary cell and edge variables together with a single-commodity flow certificate that eliminates disconnected cycles and extra components. The implementation translates this model into a PuLP/CBC solver, an independent validator, constructive instance generation, course-compatible workflow wrappers, dataset-solving tools, and a Pygame interface for visualization and interactive experimentation. Computational tests on generated instances from 5×5 to 16×16 illustrate the complete modeling-to-code pipeline and the practical role of the flow connectivity formulation.**

***Index Terms*—graph theory, mixed-integer linear programming, feasibility problem, puzzle solving, connectivity constraints, Pygame**

I. Introduction

Tracks is a logic puzzle defined on a rectangular grid. Two terminals are prescribed on the board, and the objective is to construct a railway route connecting them. The solution must satisfy two families of conditions simultaneously. On the one hand, the route must be locally and globally consistent: neighboring track pieces must match, the structure must not branch, and the final railway must form a single valid route between the prescribed terminals. On the other hand, the number of cells occupied by the route in each row and in each column must match the clues written on the boundary of the grid.

From the perspective of operations research, Tracks is naturally interpreted as a discrete feasibility problem on a graph. The combinatorial difficulty does not only come from counting constraints, but also from the global structure of the route. Local consistency constraints are not sufficient, since they allow disconnected components or isolated cycles. A rigorous model must therefore combine binary selection variables, degree conditions, and explicit connectivity constraints. This places the puzzle in the same mathematical family as several classical discrete-optimization problems in which a subset of vertices must satisfy both local incidence constraints and a global connectedness property.

This structure makes Tracks a compact case study for graph modeling. Each cell of the puzzle becomes a vertex, each possible continuation between orthogonally adjacent cells becomes an edge, and the final railway must be a simple path. Graph-theoretic notions such as degree, path, cycle, connected component, cut, and flow therefore provide a precise language for expressing the rules of the game. The use of network-flow ideas is particularly natural: flow can certify that every selected cell is reachable from a terminal, which is exactly the property that local track-piece compatibility cannot guarantee on its own [2]–[4].

The project follows this mathematical viewpoint from the model to the implementation. The grid is first represented as a graph, then the puzzle rules are translated into a mixed-integer linear formulation, and finally the formulation is implemented in Python using a standard MILP backend. The same objects also support parsing, validation, generation, dataset solving, and visualization. The purpose of the report is therefore not only to present a solver, but also to show how a visually simple puzzle can be handled as a complete graph-based feasibility problem.

II. Mathematical Model

A. Problem Statement

We consider an instance of *Tracks* on an $m \times n$ rectangular grid. Two distinguished cells are given: a start terminal and an end terminal. The objective is to determine whether one can construct a railway route joining these two terminals such that:

- each used cell belongs to the railway route,
- the route is locally consistent from one cell to the next,
- the route does not branch,
- no disconnected component or isolated cycle is present,
- the number of used cells in each row and in each column matches the given clues,
- and every fixed piece of information contained in the instance is satisfied.

The problem is therefore modeled as a pure feasibility problem. In a standard mixed-integer linear programming framework, this viewpoint can be expressed by a constant objective function,

$$\min\ 0,$$

so that any feasible solution of the model corresponds to a valid solution of the puzzle.

B. Mathematical Abstraction of the Puzzle

The puzzle is naturally represented by a graph. Each cell of the grid is associated with a vertex, and two vertices are adjacent if and only if the corresponding cells are orthogonally adjacent. This abstraction is well suited to the puzzle because the railway route is defined by local adjacencies, while its validity depends on graph-theoretic notions such as degree, path, cycle, connected component, and connectedness.

The formulation retained in this report is hybrid. Row and column clues are expressed through binary variables attached to cells, whereas continuity of the railway is described through binary variables attached to admissible edges of the grid graph. This edge-based part of the model is the natural way to express local degree conditions and the absence of branching.

We assume throughout that the intended solution is a simple path from the start terminal to the end terminal. In particular, each used non-terminal cell must have degree exactly two in the selected subgraph, each terminal must have degree exactly one, and all selected cells must belong to the same connected component. Under these assumptions, extra loops and disconnected structures are forbidden.

C. Sets, Indices, and Parameters

Let

$$R = \{1, \dots, m\} \qquad \text{and} \qquad C = \{1, \dots, n\}$$

be the sets of row and column indices. The set of cells is

$$V = R \times C.$$

The grid graph is denoted by $G = (V, E)$, where E is the set of admissible undirected edges between orthogonally adjacent cells. More precisely,

$$E = E^H \cup E^V,$$

with

$$E^H = \{\{(i,j),(i,j+1)\} : i \in R,\ j \in \{1, \dots, n-1\}\},$$

and

$$E^V = \{\{(i,j),(i+1,j)\} : i \in \{1, \dots, m-1\},\ j \in C\}.$$

For a vertex $v \in V$, we denote by $N(v)$ the set of its neighbors in G, and by

$$\delta(v) = \{e \in E : v \in e\}$$

the set of edges incident to v. For any subset $S \subseteq V$, we also use the standard graph notation

$$E(S) = \{\{u,v\} \in E : u \in S,\ v \in S\}$$

for the internal edge set of S, and

$$\delta(S) = \{\{u,v\} \in E : u \in S,\ v \notin S\}$$

for the cut induced by S.

The start and end terminals are denoted by

$$s \in V, \qquad t \in V, \qquad s \neq t.$$

The row clues are denoted by $\rho_i \in \mathbb{Z}_+$ for each $i \in R$, and the column clues by $\gamma_j \in \mathbb{Z}_+$ for each $j \in C$. Since both families of clues count the total number of used cells, a necessary consistency condition on the data is

$$\sum_{i \in R} \rho_i = \sum_{j \in C} \gamma_j.$$

To keep the model general, we also allow fixed information. Let $V^+ \subseteq V$ be the set of cells forced to belong to the route, and $V^- \subseteq V$ the set of cells forced to remain empty. Moreover, let $\mathcal{P} \subseteq V$ be the set of cells with a prescribed local track pattern. For each $v \in \mathcal{P}$, let $P_v \subseteq \delta(v)$ denote the set of incident edges that must be selected at v. In practice, P_v contains exactly two edges for a prescribed internal track piece and exactly one edge for a prescribed terminal orientation.

D. Decision Variables

For each cell $v \in V$, we introduce the binary variable

$$y_v = \begin{cases} 1 & \text{if cell } v \text{ is used by the railway route,} \\ 0 & \text{otherwise.} \end{cases}$$

For each admissible undirected edge $e \in E$, we introduce the binary variable

$$x_e = \begin{cases} 1 & \text{if edge } e \text{ is selected in the railway route,} \\ 0 & \text{otherwise.} \end{cases}$$

The cell variables are sufficient to express row and column counts, while the edge variables describe the continuity of the route. In order to enforce global connectedness, we also introduce auxiliary flow variables on directed arcs. Let

$$A = \{(u,v) \in V \times V : \{u,v\} \in E\}$$

be the set of arcs obtained by orienting each undirected edge in both directions. For each $(u,v) \in A$, we define a continuous variable

$$f_{uv} \geq 0,$$

which represents an auxiliary flow sent from the start terminal to the other selected cells.

E. Constraints

1) Domain Constraints: The variables satisfy the following domains:

$$y_v \in \{0,1\} \qquad \forall v \in V,$$

$$x_e \in \{0,1\} \qquad \forall e \in E,$$

$$f_{uv} \geq 0 \qquad \forall (u,v) \in A.$$

These constraints simply express the discrete nature of the combinatorial choices and the nonnegativity of the auxiliary flow.

2) Row and Column Count Constraints: The row clues require the number of used cells in each row to be exactly prescribed:

$$\sum_{j \in C} y_{(i,j)} = \rho_i \qquad \forall i \in R. \tag{1}$$

Similarly, the column clues impose

$$\sum_{i \in R} y_{(i,j)} = \gamma_j \qquad \forall j \in C. \tag{2}$$

These equations are the direct translation of the numerical clues of the puzzle. They count occupied cells, not track segments.

3) Consistency Between Cell and Edge Variables: If an edge is selected, then both endpoint cells must be used:

$$x_{\{u,v\}} \leq y_u \qquad \forall \{u,v\} \in E, \tag{3}$$

$$x_{\{u,v\}} \leq y_v \qquad \forall \{u,v\} \in E. \tag{4}$$

These inequalities exclude geometrically inconsistent situations in which a track connection would enter a cell not used by the route.

4) Degree Constraints and Path Structure: The terminals must belong to the route:

$$y_s = 1, \qquad y_t = 1. \tag{5}$$

Since they are the endpoints of the railway path, they must have degree exactly one in the selected subgraph:

$$\sum_{e \in \delta(s)} x_e = 1, \qquad \sum_{e \in \delta(t)} x_e = 1. \tag{6}$$

Every other used cell must have degree exactly two, whereas an unused cell must have degree zero:

$$\sum_{e \in \delta(v)} x_e = 2y_v \qquad \forall v \in V \setminus \{s,t\}. \tag{7}$$

This family of constraints is fundamental. It ensures local continuity of the route, forbids branching by excluding degree three or four, and prevents isolated used cells by excluding degree one away from the terminals. Nevertheless, degree conditions alone do not prevent the existence of several disconnected components; a global connectivity mechanism is still required.

5) Fixed and Pre-filled Information: Any cell prescribed as used is fixed by

$$y_v = 1 \qquad \forall v \in V^+, \tag{8}$$

and any cell prescribed as empty is fixed by

$$y_v = 0 \qquad \forall v \in V^-. \tag{9}$$

If the local track pattern at a cell $v \in \mathcal{P}$ is prescribed, then the corresponding incident edges are enforced by

$$x_e = 1 \qquad \forall v \in \mathcal{P},\ \forall e \in P_v, \tag{10}$$

$$x_e = 0 \qquad \forall v \in \mathcal{P},\ \forall e \in \delta(v) \setminus P_v. \tag{11}$$

These constraints make the formulation compatible with instances containing partial information, which is common in practical puzzle descriptions.

6) Connectivity Constraints: The previous constraints guarantee local validity, but they still allow an s-t path together with an additional disconnected cycle. To eliminate such invalid configurations, we impose connectedness by means of a single-commodity flow formulation. The idea is classical: the start terminal sends one unit of flow to every other selected cell, and flow can only traverse selected edges.

Let

$$M = |V| - 1$$

be a valid capacity bound. For every arc $(u,v) \in A$, flow is allowed only if the underlying undirected edge is selected:

$$0 \leq f_{uv} \leq M x_{\{u,v\}} \qquad \forall (u,v) \in A. \tag{12}$$

At the source s, the net outgoing flow equals the number of selected cells distinct from s:

$$\sum_{u \in N(s)} f_{su} - \sum_{u \in N(s)} f_{us} = \sum_{v \in V \setminus \{s\}} y_v. \tag{13}$$

For every other cell $v \neq s$, the net incoming flow must equal one unit if the cell is used, and zero otherwise:

$$\sum_{u \in N(v)} f_{uv} - \sum_{u \in N(v)} f_{vu} = y_v \qquad \forall v \in V \setminus \{s\}. \tag{14}$$

These constraints ensure that every selected cell is reachable from the start terminal by a chain of selected edges. Indeed, if a subset of selected cells were disconnected from s, then no flow could enter it because of (12), while (14) would require a positive net inflow into that component. This contradiction excludes disconnected components and isolated cycles.

7) Interpretation of the Path Constraints: Taken together, the degree and connectivity constraints characterize the required railway structure. The terminals s and t are the only selected vertices of degree one, every other selected vertex has degree two, and all selected vertices belong to one connected component. Hence the selected subgraph is a connected graph with exactly two vertices of odd degree. In this context, the only possible structure is a simple path joining s to t. Therefore, branches, detached components, and supplementary loops are all excluded by construction.

F. Objective Function and Feasibility Viewpoint

The puzzle is not associated with a meaningful cost to minimize. The role of the model is only to determine whether the set of constraints admits a feasible solution. For that reason, the complete formulation can be written as

$$\begin{aligned}
\min \quad & 0 && && \text{(15a)}\\
\text{s.t.} \quad & (1),\ (2), && &&\\
& (3),\ (4), && &&\\
& (5),\ (6),\ (7), && &&\\
& (8) - (11), && &&\\
& (12) - (14), && &&\\
& y_v \in \{0,1\} && \forall v \in V, && \text{(15b)}\\
& x_e \in \{0,1\} && \forall e \in E, && \text{(15c)}\\
& f_{uv} \geq 0 && \forall (u,v) \in A. && \text{(15d)}
\end{aligned}$$

Any feasible solution to this mixed-integer linear model is a valid solution of the corresponding Tracks instance.

G. Equivalence Between the Model and the Puzzle

The formulation above is intended to be exact with respect to the rules stated at the beginning of this section. If a feasible solution of (15) is given, the set $\{v \in V : y_v = 1\}$ defines the occupied cells and the set $\{e \in E : x_e = 1\}$ defines the railway continuations. The row and column constraints guarantee that the numerical clues are satisfied. The incidence constraints ensure that selected edges only join occupied cells. The terminal and degree constraints force the selected subgraph to have exactly two endpoints, namely s and t, and forbid branches. Finally, the flow constraints guarantee that all selected cells lie in the same connected component. Hence the selected subgraph is a single simple path from s to t, with no detached loop or extra component.

Conversely, any valid solution of the puzzle can be encoded by setting $y_v = 1$ exactly on the cells used by the railway route, $x_e = 1$ exactly on the adjacencies followed by the route, and by sending one unit of auxiliary flow from the start terminal to each other used cell along the path. This assignment satisfies every constraint of (15). Therefore, the model is not merely a relaxation of the game rules; it is a linear feasibility representation of them.

H. Modeling Remarks and Interpretation

The formulation deliberately separates the two main aspects of the puzzle. The variables y_v encode occupancy information and make it possible to express the row and column clues in a compact way. The variables x_e describe the actual geometry of the railway route through adjacency choices and degree constraints. The flow variables then provide the global certificate that the selected subgraph is connected.

The flow formulation used here is not the only possible way to impose connectivity. An alternative would be a family of cut constraints of the form

$$\sum_{e \in \delta(S)} x_e \geq y_v \qquad \forall S \subseteq V \setminus \{s\},\ \forall v \in S.$$

These constraints enforce that every selected subset not containing the source is connected to the rest of the route. They are valid, but they are exponential in number and would require an additional separation mechanism in practice. For Tracks, whose connectivity rule can be written compactly enough through a single-commodity flow model, the flow formulation is more direct and is the exact approach retained in this project.

This mathematical model therefore provides a precise and rigorous description of Tracks as a graph-based feasibility problem. It also establishes the conceptual basis for the Python implementation described in the next section.

III. Implementation Methodology

The implementation follows the mathematical model introduced in Section II. The objective is not to build an independent program with unrelated data structures, but to translate each mathematical object into a clear software object. The grid cells, admissible edges, terminal cells, row clues, column clues, and fixed information are first stored in a normalized instance representation. From this representation, the solver constructs the grid graph and then builds the mixed-integer model.

A. Software Architecture

The Python implementation is organized around four main packages. The `core` package contains the data model, parser, graph construction utilities, ASCII display tools, and independent validator. The `solver` package contains the PuLP/CBC implementation of the MILP model [5], [6]. The `generation` package creates feasible random instances by first constructing a path and then deriving row and column clues. Finally, the `ui` package contains both a lightweight Pygame viewer and a separate playable interface.

This separation mirrors the structure of the model. Instance parsing and graph construction do not depend on the solver. The validator does not depend on the MILP model. The user interface does not build mathematical constraints. As a result, the same normalized instance can be solved, validated, rendered, benchmarked, or used in an interactive game without duplicating the puzzle rules in several incompatible forms.

B. Instance Representation and File Format

The central data object is `TracksInstance`. It stores the grid dimensions, terminals, row clues, column clues, forced-used cells, forced-empty cells, forced edges, and fixed local patterns. The code uses zero-based coordinates, while the mathematical report uses one-based notation. This is only an indexing convention: a mathematical cell (i, j) corresponds to the Python tuple `(i-1, j-1)`.

The text format used for instances is deliberately simple. Required fields are `rows`, `cols`, `start`, `end`, `row_clues`, and `col_clues`. Optional fields encode fixed information. The field `fixed_used` forces a cell to be occupied, but does not prescribe its shape. The field `fixed_empty` forbids a cell from being used. The field `fixed_edges` forces a connection between two adjacent cells. The field `fixed_patterns` is stronger: it fixes the exact local directions of the track piece in a cell. During normalization, terminals are automatically treated as used cells, undirected edges are canonicalized, and inconsistent hints are rejected.

C. Graph Construction and Structural Validation

The graph layer constructs the objects used in the mathematical formulation: the vertex set V, horizontal and vertical edge sets E^H and E^V, the complete edge set E, neighbor sets $N(v)$, incident edge sets $\delta(v)$, and the directed arc set A. This layer is deterministic and pure: it only derives graph objects from the normalized instance.

The independent validator is a second line of defense. After a candidate solution is produced, either by the solver or by the playable interface, the validator checks the row and column counts, terminal degrees, internal degrees, fixed

hints, connectedness, absence of branches, and absence of disconnected cycles. This is important for an optimization project because the solver output should not be trusted solely on the basis of solver status. A solution is accepted by the application only when it also satisfies the structural validator.

D. MILP Translation

The MILP builder translates the constraint families of Section II directly into PuLP expressions. Cell variables y_v, edge variables x_e, and flow variables f_{uv} are created over the grid graph. Row and column constraints are generated by summing the corresponding cell variables. Edge-cell consistency constraints link each x_e to its endpoint variables. Degree constraints impose the path structure. Fixed information is translated into variable-fixing constraints. Finally, flow-capacity and flow-balance constraints impose global connectedness.

The optimization objective is the constant value 0, because the puzzle is a feasibility problem rather than a cost-minimization problem. The MILP solver therefore searches for any feasible assignment satisfying all constraints. If PuLP/CBC reports an optimal or feasible assignment, the selected cells and edges are decoded into a `TracksSolution` object and passed to the validator.

E. Constructive Instance Generation

The generator avoids producing arbitrary clue systems. Instead, it first constructs a simple path between the terminals. Once this path has been found, the row and column clues are computed from the cells visited by the path. This guarantees that every generated instance has at least one feasible solution. Optional hints are then sampled from the path as forced-used cells or forced edges.

This procedure does not guarantee uniqueness, and uniqueness is not required for the Tracks part of the project. Its purpose is to provide controlled feasible instances for testing, benchmarking, and interactive play. Larger instances can be generated by increasing the grid size or by requiring longer paths.

F. Course-Compatible Workflow

The course reference workflow is expressed through functions such as `readInputFile`, `cplexSolve`, `displayGrid`, `displaySolution`, `generateDataSet`, `solveDataSet`, and `resultsArray`. The project is implemented in Python rather than Julia, but equivalent wrappers are provided in `tracks_solver.course_workflow`. The name `cplexSolve` is kept as a course-compatible alias even though the backend used here is PuLP/CBC.

The workflow can therefore read a manual instance, display it, solve it with the MILP backend, display the solution, generate random datasets, solve a folder of instances, and export result files containing `solveTime` and `isOptimal`. These artifacts make the project easier to compare with the expected course structure while preserving the Python architecture.

Table I
GENERATED BENCHMARK PROTOCOL.

Item	Value
Instance source	Constructive generator only
Saved location	`data/tracks/generated/`
Board sizes	$5 \times 5, 6 \times 6, \ldots, 16 \times 16$
Difficulties	Easy, Medium, Hard
Attempts per configuration	10
Total instances	360
Solver backend	PuLP/CBC exact-flow MILP
Time limit per instance	120 s
Result file	`res/tracks/report/generated_benchmark_results.csv`

Table II
AGGREGATE OUTCOME OVER THE GENERATED BENCHMARK.

Difficulty	Instances	Solved	Avg. solve (s)	Avg. hints
Easy	120	120	0.078	28.5
Medium	120	120	0.127	15.8
Hard	120	120	0.148	8.3

G. Visualization and Playable Interface

The Pygame layer is separated from the solver. The basic viewer displays an instance and, when available, an exact solution returned by the MILP backend. The playable interface adds a menu, random-game generation, map loading, player editing, explicit validation, and solution reveal. Fixed hints are rendered with different backgrounds: blocked cells, required cells, forced connections, and exact fixed patterns are visually distinguished.

The UI is not required for the mathematical correctness of the solver, but it is useful for interpreting instances and for defense demonstrations. In particular, it makes visible the difference between local track placement and global graph validity: a visually plausible partial route may still fail the degree, clue, or connectivity checks.

IV. COMPUTATIONAL RESULTS

The computational study focuses on generated Tracks instances only. This choice isolates the behavior of the exact MILP formulation from hand-crafted or screenshot-derived boards, and it gives a controlled benchmark in which the grid size and the generation difficulty are varied systematically. Every instance is generated, stored as a text file, solved with the same PuLP/CBC backend, and then checked with the independent validator.

Table I gives the complete experimental protocol. The benchmark contains

$$12 \text{ sizes} \times 3 \text{ difficulties} \times 10 \text{ attempts} = 360$$

instances. For each row, the collector records the generated file path, generation time, solver time, solver status, validator status, route length, number of fixed hints, and the time limit. The generated files are kept in the repository data folder so that the experiment can be inspected or rerun without silently replacing the underlying boards.

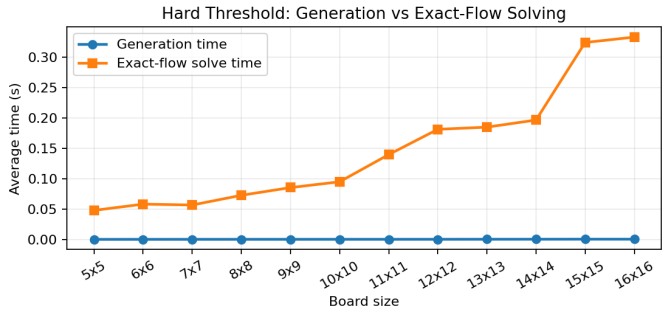

Figure 1. Generation time and exact-flow solve time for Hard generated instances as the board size increases.

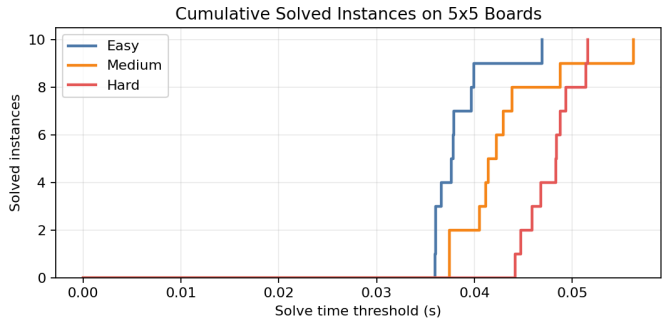

Figure 2. Cumulative number of solved generated instances as a function of time for 5×5 boards.

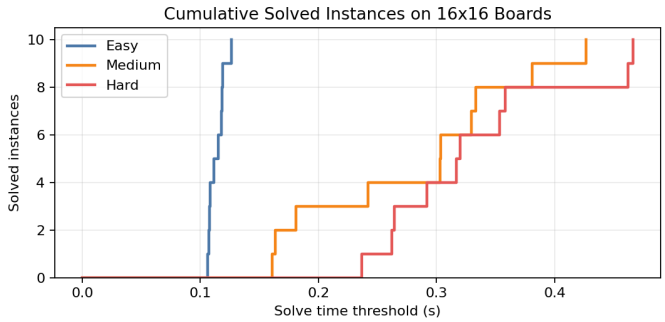

Figure 3. Cumulative number of solved generated instances as a function of time for 16×16 boards.

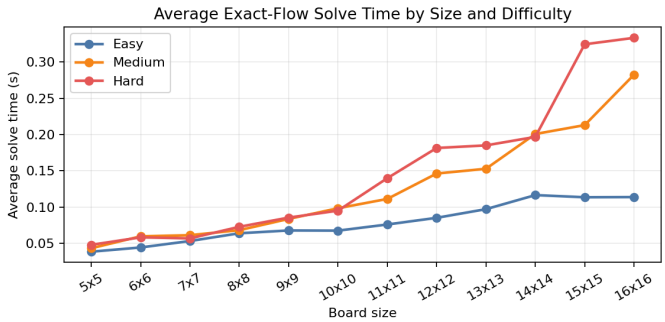

Figure 4. Average exact-flow solution time as a function of board size for each generation difficulty.

Table II summarizes the main numerical outcome. All 360 generated instances were solved to optimality and passed the validator. The average solve time increases from Easy to Hard because the generator requests longer paths and exposes fewer fixed hints. This behavior is consistent with the graph model: longer paths use more vertices and edges, and fewer hints leave more feasible local configurations for the MILP to separate through the global flow constraints.

Fig. 1 compares the two costs involved in the empirical Hard profile: constructing a valid instance and solving it exactly with the flow MILP. Generation remains almost negligible because the generator first builds a valid path and then derives row and column clues from it. The solve time is larger and grows with the board size, from approximately 0.048 s on 5×5 boards to approximately 0.333 s on 16×16 boards. This confirms that the exact connectivity model is the dominant computational component of the pipeline.

The cumulative curve in Fig. 2 shows the small-board regime. All 5×5 instances are solved within a fraction of a second, independently of the difficulty profile. The separation between Easy, Medium, and Hard is therefore visible but not critical at this size: the graph has only 25 vertices, and the number of binary variables and flow arcs remains small.

Fig. 3 gives the same view for the largest boards in the benchmark. The three difficulty profiles remain solvable within the time limit, but their curves spread more clearly. Easy instances finish first because their paths are shorter and their hints are denser. Medium and Hard instances require more time because the solver must certify a longer connected route over a larger graph with fewer prescribed local decisions.

Fig. 4 is the main scalability plot. The average solution time increases with size for all three difficulty levels, and the Hard curve is generally above the Easy curve. The largest observed averages are about 0.114 s for Easy 16×16, 0.282 s for Medium 16×16, and 0.333 s for Hard 16×16. The trend is not perfectly monotone at every intermediate size because each point averages only ten generated boards, but the overall pattern is clear: larger graphs and fewer fixed hints produce larger MILP instances and longer exact-flow solving times.

These experiments support two conclusions. First, the implementation of the mathematical model is consistent: every generated board is solved, decoded, and validated according to the row clues, column clues, local degree rules, fixed information, and global connectedness condition. Second, within the tested range, the flow-based MILP is practical for generated Tracks boards up to 16×16. All reported solutions come from the exact feasibility formulation described in Section II.

V. Conclusion

This project models *Tracks* as a graph-based feasibility problem and implements the resulting formulation in Python. The grid is represented as an undirected graph, the railway route is described through selected vertices and edges, and the path structure is enforced by degree constraints together with a single-commodity flow formulation. This combination

is essential: local degree rules prevent branches, while the flow certificate eliminates disconnected cycles and extra components.

The implementation follows the mathematical formulation closely. Instance files are parsed into normalized data objects, graph helpers construct the sets used by the MILP, the PuLP/CBC backend solves the feasibility model, and an independent validator checks the returned solution. Additional tools make the project compatible with the course workflow by supporting manual test instances, dataset generation, batch solving, result files, and LaTeX result tables. The Pygame interface provides a visual layer for inspecting solved instances and for experimenting with playable boards.

The computational experiments show that the solver behaves correctly on a controlled generated benchmark from 5×5 to 16×16. All generated instances in the benchmark are solved to optimality and validated, while the average solving time increases with the board size and with the difficulty profile. This confirms both the practical usefulness of the exact model and the expected growth in difficulty as the graph and route structure become more complex.

Several extensions remain possible. The generated benchmark could be expanded to larger grids and more difficulty profiles. The visual interface could also be improved with richer interaction tools and better puzzle-selection features. These extensions would make it possible to study the practical limits of the exact flow formulation while keeping the project focused on the Tracks model actually required here.

References

[1] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[2] H. Validi, A. Buchanan, and E. Lykhovyd, "Imposing contiguity constraints in political districting models," *Operations Research*, vol. 70, no. 2, pp. 867–892, 2022, doi: 10.1287/opre.2021.2141.

[3] F. K. Miyazawa, P. F. S. Moura, M. J. Ota, and Y. Wakabayashi, "Partitioning a graph into balanced connected classes: Formulations, separation and experiments," *European Journal of Operational Research*, vol. 293, no. 3, pp. 826–836, 2021, doi: 10.1016/j.ejor.2020.12.059.

[4] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.

[5] Python Software Foundation, "Python Language Reference," 2026. [Online]. Available: https://www.python.org

[6] S. Mitchell, M. O'Sullivan, and I. Dunning, "PuLP: A linear programming toolkit for Python," 2011.